VELLORE INSTITUTE OF TECHNOLOGY (VIT), VELLORE

School of Computer Science and Engineering (SCOPE)
Department of Computer Science and Engineering

FINAL PROJECT REPORT
BCSE302P — Database Systems Lab

Compression-Aware Vector Database Indexing for Memory-Efficient and Low-Latency Retrieval-Augmented Generation (RAG)

Evaluating Half-Precision (halfvec) Coordinate Representation, PostgreSQL TOAST Storage Behaviour, and HNSW Graph Indexing in pgvector

Project Details	Official Information
Programme & Branch	B.Tech. in Computer Science and Engineering
School & Institution	School of Computer Science and Engineering (SCOPE), VIT Vellore
Course Code & Title	BCSE302P — Database Systems Lab
Academic Semester & Slot	Fall Semester 2026–27 | Slot: L37+L38
Team Member 1	Yash Pradhan | Registration No.: 24BCE0702
Team Member 2	Anjini Pandey | Registration No.: 24BCE0714
Faculty Supervisor	Dr. D. Anuradha, School of Computer Science and Engineering, VIT Vellore
Evaluated Database Stack	PostgreSQL 18.4 with pgvector 0.8.4 (nomic-embed-text, 768 dimensions)
Student Signatures	_____ [Signature — Yash Pradhan] _____ [Signature — Anjini Pandey]
Faculty Signature & Date	_____ [Signature — Dr. D. Anuradha] Date: ____ / ____ / _____ [Date of Submission]

Acknowledgement

We express our sincere gratitude to our faculty supervisor, **Dr. D. Anuradha**, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, for her guidance, constructive feedback across Project Reviews 1 and 2, and continuous encouragement throughout the BCSE302P Database Systems Lab project. We also thank the School of Computer Science and Engineering at VIT Vellore for providing the academic environment needed to carry out this work, as well as the open-source maintainers of PostgreSQL, pgvector, Ollama, and the BeIR/msmarco dataset whose software and data resources made our empirical evaluation possible.

Authors: Yash Pradhan (24BCE0702) & Anjini Pandey (24BCE0714)

Abstract

Retrieval-Augmented Generation (RAG) systems often use PostgreSQL with the pgvector extension so that relational metadata and dense vector embeddings can be stored and queried together. A practical storage problem arises when storing 768-dimensional 32-bit floating-point embeddings (vector(768)) generated by models such as nomic-embed-text. In PostgreSQL, each 768-dimensional FP32 vector occupies 3,076 bytes as measured empirically by pg_column_size(embedding) (compared with 3,080 bytes under the documented 4d + 8 formula). Because this exceeds PostgreSQL's ~2 KiB TOAST_TUPLE_THRESHOLD on standard 8 KiB pages and dense float arrays do not compress well under byte-level LZ algorithms, PostgreSQL moves every FP32 vector out-of-line into secondary TOAST (pg_toast) tables. During sequential scans, reconstructing these out-of-line chunks substantially increases shared-buffer page accesses.

In this project, we evaluate 16-bit half-precision coordinate quantization (halfvec(768)) combined with Hierarchical Navigable Small World (HNSW) graph indexing in PostgreSQL 18.4 and pgvector 0.8.4 across BeIR/msmarco subsets of 180 chunks (Review 2 baseline), 1,001 chunks (Stages 1–3), 5,000 chunks, and 10,000 chunks (Stage 4 v2). Casting embeddings to FP16 reduces the measured column size by 49.80% to 1,544 bytes (49.87% by formula), allowing vectors to stay inline within main heap pages and eliminating 100% of out-of-line TOAST data payloads (0 bytes, with only an empty 8 KiB TOAST index shell remaining). Across our dedicated vector-only representation tables, pre-HNSW base relation size drops by 60.11% at 1,001 chunks, inclusive post-index relation size drops by 55.35% at 5,000 chunks and 55.37% at 10,000 chunks, and standalone HNSW index size drops by 49.90%–49.98%. Across 100 timed trials per configuration in Stage 4 v2, FP16 achieves a 6.22x lower mean latency (7.12x lower median) in exact sequential search at 5,000 chunks and a 7.24x lower mean latency (8.03x lower median) at 10,000 chunks—an observation consistent with 93.35% fewer Query 0 (Q0) diagnostic shared-buffer hits—while keeping forced-HNSW latency broadly similar (~3.37–4.13 ms mean), preserving 0.9600 Recall@10 against FP32 exact ground truth, and maintaining 0.9500–0.9850 pairwise top-10 candidate agreement.

Keywords: Retrieval-Augmented Generation (RAG); PostgreSQL; pgvector; Half-Precision Quantization (halfvec); TOAST Storage; HNSW Indexing; Shared-Buffer Access; Recall@K.

Executive Summary

When building Retrieval-Augmented Generation (RAG) applications on top of PostgreSQL and pgvector, developers typically store 768-dimensional text embeddings using the default 32-bit vector(768) data type. Because each 32-bit embedding takes roughly 3 KiB (3,076 bytes measured), it is too large to sit comfortably inside PostgreSQL's 8 KiB table pages (~2 KiB TOAST threshold). As a result, PostgreSQL pushes every vector out of the main table into a separate TOAST table on disk and in the buffer pool. This project investigates a practical database engineering question: what happens to table storage, index size, query latency, and search accuracy when we store those same 768-dimensional embeddings as 16-bit half-precision floats (halfvec(768), 1,544 bytes)?

- **Work Completed:** Across four structured stages (expanding from a 180-chunk baseline in Review 2 to 1,001, 5,000, and 10,000 BeIR/msmarco chunks supported by a 43-test automated pytest suite), we built isolated vector-only tables for FP32 and FP16, constructed HNSW graph indexes (m = 16, ef_construction = 64), and benchmarked 100 timed trials per configuration across 20 fixed queries.
- **Main Findings:** Converting to halfvec(768) kept all embeddings inline inside the main table heap, reducing out-of-line TOAST data payloads to 0 bytes (with only an empty 8 KiB TOAST index shell remaining). Total post-index relation storage dropped by **55.35%–55.37%** at 5,000 and 10,000 chunks, and HNSW index size dropped by **49.90%–49.98%**. In exhaustive exact sequential search, the inline FP16 representation achieved a **6.22x–7.24x lower mean latency (7.12x–8.03x lower median latency)** than FP32—an observation consistent with the **93.35% fewer** shared-buffer hits recorded in the single-query Q0 diagnostic—while keeping in-memory HNSW traversal times broadly similar (~3.37–4.13 ms mean) and retaining **0.9600 Recall@10** against uncompressed FP32 exact search.
- **Practical Conclusion:** For 768-dimensional embeddings in PostgreSQL, halfvec(768) cuts total table-plus-index storage by more than half and substantially reduces measured exact-search latency without sacrificing top-10 HNSW retrieval accuracy.

Table of Contents

Acknowledgement ..... **ii**
Abstract ..... **ii**
Executive Summary ..... **iii**
List of Figures ..... **v**
List of Tables ..... **v**
Abbreviations ..... **vi**
Symbols and Notations ..... **vi**
1. INTRODUCTION **1**
1.1 BACKGROUND 1
1.2 MOTIVATIONS 1
1.3 SCOPE OF THE PROJECT 1
2. PROJECT DESCRIPTION AND GOALS **2**
2.1 LITERATURE REVIEW 2
2.2 RESEARCH GAP 3
2.3 OBJECTIVES 4
2.4 PROBLEM STATEMENT 4
2.5 PROJECT PLAN 4
3. TECHNICAL SPECIFICATION **5**
3.1 REQUIREMENTS 5
3.1.1 Functional 5
3.1.2 Non-Functional 6
3.2 FEASIBILITY STUDY 6
3.2.1 Technical Feasibility 6
3.2.2 Economic Feasibility 7
3.2.3 Social Feasibility 7
3.3 SYSTEM SPECIFICATION 7
3.3.1 Hardware Specification 7
3.3.2 Software Specification 7
4. DESIGN APPROACH AND DETAILS **8**
4.1 SYSTEM ARCHITECTURE 8
4.2 DESIGN 9
4.2.1 Data Flow Diagram 9
4.2.2 Class Diagram 10
5. METHODOLOGY AND TESTING **10**
5.1 MODULE DESCRIPTION 10
5.2 TESTING 11
6. PROJECT DEMONSTRATION **12**
6.1 REPRESENTATION TABLE DEFINITIONS AND HALF-PRECISION CASTING 12
6.2 EXACT-SEARCH AND HNSW RETRIEVAL QUERY PATTERNS 13
6.3 BENCHMARK TIMING AND QUERY-PLAN VERIFICATION 14
7. RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE) **14**
7.1 PHYSICAL STORAGE, TOAST BEHAVIOUR, AND HNSW BUILD-TIME OBSERVATIONS 14
7.2 EXACT-SEARCH AND FORCED-HNSW LATENCY COMPARISONS 16
7.3 RETRIEVAL FIDELITY: RECALL@K AND PAIRWISE CANDIDATE-SET AGREEMENT 18
7.4 QUERY-PLAN, SHARED-BUFFER DIAGNOSTICS, AND PLANNER BEHAVIOUR 20
7.5 RESOURCE COST ANALYSIS, PRACTICAL IMPLICATIONS, AND LIMITATIONS 23
8. CONCLUSION AND FUTURE ENHANCEMENTS **24**
8.1 CONCLUSION 24
8.2 FUTURE ENHANCEMENTS 24
9. REFERENCES **25**
APPENDIX A — SAMPLE CODE **26**
APPENDIX B — TANGIBLE OUTCOME — PUBLICATION/PATENT **27**

List of Figures

Figure 1: Implemented system architecture and benchmarking pipeline....8
Figure 2: Level-1 Data Flow Diagram (DFD) of the implemented system....10
Figure 3: UML database entity-relationship and procedural module diagram....10
Figure 4: Physical storage footprint across 1,001, 5,000, and 10,000 chunks....16
Figure 5: Client-observed retrieval latency under Stage 4 v2 (chart4_latency.csv)....18
Figure 6: Representation-specific ANN Recall@K (zero-based 0.0–1.0 axis)....19
Figure 7: Pairwise top-K candidate agreement between FP32 and FP16 HNSW....20
Figure 8: Single-query Query 0 (Q0) shared-buffer and execution diagnostics....22

List of Tables

Table 2.1: Comparison of FP32, FP16 halfvec, and codebook quantization....3
Table 2.2: Project execution plan and milestone progression across stages....5
Table 3.1: Functional and non-functional system requirements....6
Table 3.2: Experimental hardware and software specifications....7
Table 4.1: Multi-database layout and isolation policy across stages....9
Table 5.1: Functional responsibilities of Python modules in src/....11
Table 5.2: Automated pytest suites and timing boundaries across stages....12
Table 7.1: Coordinate datum size comparison for 768-dimensional vectors....14
Table 7.2: Disaggregated storage footprint and HNSW build durations....15
Table 7.3: Stage 4 v2 client-observed latency at 5,000 and 10,000 chunks....17
Table 7.4: Representation-specific Recall@K against FP32 exact ground truth....19
Table 7.5: Pairwise top-K candidate agreement between FP32 and FP16 HNSW....20
Table 7.6: Single-query Query 0 (Q0) EXPLAIN (ANALYZE, BUFFERS) diagnostics....21
Table 7.7: PostgreSQL query optimizer cost estimates and plan selection....22

Abbreviations

Abbrev.	Full Term	Abbrev.	Full Term
ACID	Atomicity, Consistency, Isolation, Durability	IQR	Interquartile Range (P75 - P25)
ANN	Approximate Nearest Neighbor	KiB / MiB	Kibibyte (1,024 B) / Mebibyte (1,048,576 B)
BLCKSZ	PostgreSQL Block Size (8,192 B / 8 KiB)	KNN	k-Nearest Neighbors
DDL / DML	Data Definition / Manipulation Language	LLM	Large Language Model
DFD	Data Flow Diagram	MS MARCO	Microsoft Machine Reading Comprehension
FP16 / halfvec	IEEE 754 16-Bit Half-Precision Float	NPU	Neural Processing Unit
FP32 / vector	IEEE 754 32-Bit Single-Precision Float	PQ / SQ	Product / Scalar Quantization
FSM / VM	Free Space Map / Visibility Map Forks	RAG	Retrieval-Augmented Generation
GUC	Grand Unified Configuration (PostgreSQL)	RDBMS	Relational Database Management System
HNSW	Hierarchical Navigable Small World Graph	TOAST	The Oversized-Attribute Storage Technique
IEEE	Institute of Electrical & Electronics Eng.	WAL	Write-Ahead Log

Symbols and Notations

Symbol / Notation	Mathematical or Database Meaning
d ; u, v in R^d	Embedding dimensionality (d = 768) and 768-D chunk/query vectors
d_cos(u, v) / <=>	Cosine distance operator in pgvector: 1 - (u . v) / (||u||_2 ||v||_2)
N ; K	Relation chunk count (180, 1,001, 5,000, 10,000) and top-K cutoff (K in {3, 5, 10})
m	Maximum bidirectional HNSW links per node on upper graph layers (m = 16)
ef_construction	Dynamic candidate list size during HNSW index build (ef_construction = 64)
ef_search	Dynamic candidate priority-queue size during HNSW query search (ef_search = 40)
Top-K_FP32_Exact(q)	Ground-truth top-K chunk ID set from exhaustive FP32 exact scan for query q
S_FP32(q), S_FP16(q)	Top-K candidate ID sets returned by FP32 HNSW and FP16 HNSW for query q
Recall@K	Mean top-K overlap fraction |Top-K_FP32_Exact(q) intersect Top-K_Cand(q)| / K
Agreement@K	Mean pairwise top-K overlap fraction |S_FP32(q) intersect S_FP16(q)| / K

1. INTRODUCTION

1.1 BACKGROUND

Retrieval-Augmented Generation (RAG) grounds Large Language Model (LLM) responses by coupling a parametric sequence-to-sequence generator with a non-parametric dense-vector memory over an external document corpus [10]. To support semantic retrieval, document passages are segmented into chunks and mapped into high-dimensional vectors using neural embedding models such as nomic-embed-text (768 dimensions) [4]. Rather than maintaining a separate standalone vector store alongside a relational database, many production systems use PostgreSQL with the pgvector extension [2] so that document text, foreign-key metadata, and vector indexes share a single ACID-compliant storage engine. However, PostgreSQL organizes relation data into fixed-size 8 KiB pages (BLCKSZ = 8192) designed for compact scalar tuples rather than multi-kilobyte floating-point coordinate arrays [1].

1.2 MOTIVATIONS

When 768-dimensional embeddings are stored in standard 32-bit single-precision format (vector(768)), each vector datum occupies 3,076 bytes as measured empirically by pg_column_size(embedding) (3,080 bytes under the documented 4d + 8 formula). Because this exceeds PostgreSQL's ~2 KiB oversized-attribute threshold (TOAST_TUPLE_THRESHOLD), PostgreSQL moves every FP32 vector out of the main table heap into a secondary TOAST (pg_toast) table [1]. This out-of-line storage creates two practical problems: secondary TOAST tables and 32-bit HNSW indexes consume substantial disk and memory space, and exhaustive sequential scans must follow pointers and reassemble TOAST chunks for every row.

Converting coordinates from 32-bit floats (vector(768)) to 16-bit half-precision floats (halfvec(768)) cuts the vector datum size in half to 1,544 bytes. Because 1,544 bytes sits safely below the ~2 KiB TOAST threshold, PostgreSQL stores the entire vector inline inside the main 8 KiB heap page. Our motivation is to measure how much storage, buffer work, and query latency are saved when halfvec(768) is used across both exact sequential scans and HNSW approximate indexes, and whether 16-bit coordinate precision sacrifices top-K retrieval accuracy.

1.3 SCOPE OF THE PROJECT

This project evaluates the database storage and vector retrieval layer in PostgreSQL 18.4 with pgvector 0.8.4 across BeIR/msmarco subsets [3] of 180 chunks (Review 2 baseline on a monolithic table) and 1,001, 5,000, and 10,000 chunks (Stages 1–4 on dedicated vector-only representation tables). We measure disaggregated physical storage (heap, TOAST data, TOAST index, primary-key index, HNSW index, and auxiliary forks), 100-trial client retrieval latency across Exact Search, Forced HNSW, and Planner-Selected modes, single-query EXPLAIN (ANALYZE, BUFFERS) diagnostics, planner path selection, ground-truth Recall@K, and pairwise top-K candidate agreement. We do not claim to measure downstream LLM answer generation quality, human relevance judgments (qrels), cold-disk paging beyond PostgreSQL's 128 MB shared_buffers, or general asymptotic scaling laws beyond the tested 5,000- and 10,000-chunk Stage 4 v2 sizes.

2. PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

Our investigation connects peer-reviewed research on retrieval-augmented generation, graph-based approximate nearest-neighbor (ANN) search, vector compression, and ANN benchmarking with the physical storage mechanics of PostgreSQL and pgvector:

- **RAG and Dense Vector Retrieval [10, 3, 4]:** Lewis et al. [10] formulated Retrieval-Augmented Generation (RAG) for knowledge-intensive NLP tasks by combining a pre-trained parametric generator with a non-parametric dense-vector retriever indexed over external text passages. Because the retriever must identify the top-K highest-similarity passages from large corpora such as MS MARCO [3] using dense embeddings (in our pipeline, 768-dimensional nomic-embed-text vectors [4]), the storage footprint, query latency, and top-K recall of the vector index directly govern retrieval efficiency.
- **Graph-Based Approximate Search via HNSW [6, 2]:** To avoid exhaustive O(N d) linear scans over all N vectors, Malkov and Yashunin [6] introduced Hierarchical Navigable Small World (HNSW) graphs, which organize vectors into nested proximity graphs with exponentially decaying node density across upper routing layers and a dense base layer (Layer 0). Greedy search descends from coarse upper-layer entry links to fine-grained Layer-0 neighborhoods using three core hyperparameters—maximum links per node (m), build-time candidate list size (ef_construction), and query-time priority-queue width (ef_search) [6]—which are implemented directly in pgvector (m = 16, ef_construction = 64, ef_search = 40 in our experiments) [2].
- **Vector Compression and Quantization [7, 8, 2]:** Storing uncompressed 32-bit floating-point coordinates in memory and on disk creates substantial capacity and bandwidth costs as vector collections grow. Jégou et al. [7] introduced Product Quantization (PQ), which partitions a d-dimensional vector into M orthogonal subvectors (distinct from the HNSW graph degree parameter m) and quantizes each subspace using a learned codebook of 2^b centroids (b bits per subquantizer, yielding an M x b-bit code) with distances estimated via lookup tables. Whereas PQ and Scalar Quantization (SQ) define compact code formats, Guo et al. [8] formulated anisotropic vector quantization as a quantization loss objective tailored to maximum inner-product and cosine similarity search—penalizing reconstruction error parallel to a vector more heavily than orthogonal error when optimizing codebooks so that relative similarity rankings are better preserved. Although learned codebook methods [7, 8] produce compact codes whose length depends on the chosen M and b, they require offline training on representative data; while a fixed codebook does not itself change when new SQL rows are inserted, incoming data whose distribution shifts away from the training sample may become less well represented by the existing centroids, potentially motivating recalibration or retraining. In contrast, coordinate-level IEEE 754 half-precision quantization (halfvec(768) in pgvector [2]) deterministically casts each 32-bit coordinate to a 16-bit binary16 float (1,544 B datum size vs. 3,076 B measured for vector(768)) without codebook training, while supporting native HNSW indexing (halfvec_cosine_ops) (Table 2.1).
- **ANN Benchmarking vs. Relational Storage Mechanics [9, 1, 2]:** Aumüller et al. [9] established the ANN-Benchmarks framework, demonstrating that approximate search methods must be evaluated across multiple competing dimensions—specifically top-K recall against exact ground truth (Recall@K), query latency/throughput, and index build cost. However, ANN-Benchmarks

[9] evaluates standalone in-memory indexing libraries in isolation from relational database page management. Inside PostgreSQL [1], relation pages are fixed at 8,192 bytes (BLCKSZ = 8192), and tuples exceeding TOAST_TUPLE_THRESHOLD (~2 KiB) that cannot be compressed below TOAST_TUPLE_TARGET (~2 KiB) by byte-level LZ algorithms (pglz/lz4) are sliced into out-of-line pg_toast chunks. Consequently, storing vector(768) (3,076 B measured) triggers out-of-line TOAST eviction, whereas halfvec(768) (1,544 B) stays inline within primary 8 KiB heap pages [1, 2].

Table 2.1: Comparison of FP32, FP16 halfvec, and codebook quantization

Comparison Aspect	FP32 Baseline (vector(768)) [2]	FP16 Quantization (halfvec(768)) [2]	Learned Codebook & Scalar Quantization (PQ [7], SQ; Anisotropic Objective [8])
Encoding & Design Role	IEEE 754 32-bit float (4 B/dim)	IEEE 754 16-bit float (2 B/dim)	PQ: M subquantizers x b-bit learned codebooks [7]; SQ: scalar buckets (anisotropic loss [8] defines training objective)
768-D Datum Footprint	3,076 B measured (3,080 B formula)	1,544 B (49.80% empirical reduction)	Configuration-dependent: M x b bits for PQ [7] (depends on chosen subquantizers M and bits per code b)
PostgreSQL Heap / TOAST [1]	Evicted out-of-line to pg_toast	Inlined in 8 KiB heap page (< ~2 KiB)	Inlined if encoded representation is below ~2 KiB threshold
Training & Distribution Shift	None (native SQL inserts)	None (deterministic SQL cast)	Requires offline codebook/range fitting [7, 8]; distribution shift in new data may motivate retraining
Evaluated in This Project	Yes — Stages 1–4 Baseline	Yes — Stages 2–4 Primary Target	Reviewed as literature context [7, 8]; not implemented in benchmark

2.2 RESEARCH GAP

The foundational literature reviewed above investigates RAG architecture and dense retrieval [10], HNSW proximity-graph construction and search [6], learned subspace and anisotropic vector quantization [7, 8], and standardized in-memory ANN benchmarking [9], while official PostgreSQL and pgvector documentation defines page storage, TOAST thresholds, and vector/halfvec data types [1, 2]. However, standalone ANN benchmarks [9] and algorithmic quantization studies [7, 8] do not directly measure how coordinate precision interacts with PostgreSQL's 8 KiB page layout, out-of-line pg_toast storage, shared-buffer access patterns, and query-planner cost model.

Within the literature reviewed, our project addresses a focused database-engineering setting: empirically comparing vector(768) (FP32) and halfvec(768) (FP16) inside PostgreSQL 18.4 and pgvector 0.8.4 on dedicated vector-only representation tables. Specifically, our experiments measure how half-precision representation affects (1) disaggregated heap, TOAST data, TOAST

index, and auxiliary fork storage, (2) standalone HNSW index footprint and single-run build duration, (3) 100-trial client retrieval latency across exact sequential scans, forced HNSW scans, and planner-selected execution, (4) single-query Q0 shared-buffer hits and representation-specific planner transitions between Seq Scan and Index Scan, and (5) representation-specific Recall@K and pairwise top-K candidate agreement across 1,001, 5,000, and 10,000 MS MARCO chunks.

2.3 OBJECTIVES

- **1. Disaggregated Storage Auditing:** Quantify byte and MiB footprints across datums, main heaps, TOAST data tables, TOAST indexes, primary-key indexes, HNSW indexes, and auxiliary forks (_fsm, _vm) at 1,001, 5,000, and 10,000 chunks.
- **2. Controlled Latency Benchmarking:** Measure FP32 and FP16 client retrieval latency across Exact Search (Seq Scan), Forced HNSW (Index Scan, ef_search = 40), and Planner-Selected modes over 20 pre-embedded queries, 2 warm-ups, and 5 alternating passes (100 timed trials per configuration).
- **3. Retrieval Fidelity & Agreement Analysis:** Evaluate representation-specific Recall@K (K in {3, 5, 10}) against uncompressed FP32 exact ground truth alongside pairwise top-K candidate agreement between FP32 HNSW and FP16 HNSW.
- **4. Query-Plan & Optimizer Cost Inspection:** Inspect single-query EXPLAIN (ANALYZE, BUFFERS) plans on Query 0 (Q0) to record shared-buffer hits, server execution times, and planner cost transitions.

2.4 PROBLEM STATEMENT

> **Problem Statement**
>
> Storing 768-dimensional FP32 vectors (vector(768), 3,076 B measured) in PostgreSQL exceeds the ~2 KiB TOAST threshold, forcing out-of-line pg_toast storage and doubling HNSW index size relative to 16-bit floats. This project measures whether half-precision coordinate representation (halfvec(768), 1,544 B) eliminates out-of-line TOAST data payloads, reduces base and HNSW index storage, improves exact or indexed query latency, and preserves top-K nearest-neighbor overlap across 1,001, 5,000, and 10,000 MS MARCO text chunks.

2.5 PROJECT PLAN

Table 2.2 summarizes the actual stage-by-stage progression of the project from initial problem formulation in Review 1 through the completed Stage 4 scaling experiments and final synthesis.

Table 2.2: Project execution plan and milestone progression across reviews and experimental stages.

Project Phase / Milestone	Scope & Corpus Scale	Key Technical Work Completed & Verified Outcome
Review 1 Milestone	Problem Formulation	Defined the four-way RAG trade-off (memory, storage, latency, Recall@K) and surveyed ANN indexing and vector compression.
Review 2 Prototype & Stabilized Baseline	100 docs / 180 chunks (compression_aware_rag)	Implemented ingestion, chunking (300/50), Ollama nomic-embed-text embeddings, and stabilized 180-chunk baseline (2.180 ms exact vs. 2.270 ms HNSW; 11/11 pytest).
Stage 1: Dataset Scaling & Atomic Caching	550 docs / 1,001 chunks (compression_aware_rag_exp)	Built deterministic HF streaming downloader with atomic .tsv.tmp caching and JSON provenance manifest (msmarco_1k_chunks.meta.json; 19/19 pytest).
Stage 2: FP32 vs. FP16 Exact Evaluation	550 docs / 1,001 chunks (Dedicated vector tables)	Created chunks_embedding_fp32 and chunks_embedding_halfvec; verified 60.11% base storage reduction, 0 B TOAST data on FP16, and 1.86x exact speedup (27/27 pytest).
Stage 3: 1k HNSW Indexing & Plan Check	550 docs / 1,001 chunks (HNSW m=16, ef_c=64)	Built FP32 and FP16 HNSW indexes (49.90% index reduction); evaluated Forced vs. Planner-Selected modes and 1.0000 Recall@3/5/10 (34/34 pytest).
Stage 4: 5k & 10k Scaling (v1 -> v2 Audit)	2,729 docs (5k) & 5,457 docs (10k) (..._scale)	Validated 1,001/1,001 SHA-256 key-set match; ran 6-config Stage 4 v2 benchmark (100 trials/config, GUCs outside timer) and fork-reconciled storage audit (43/43 pytest).
Final Synthesis & Report	Cross-Stage Audit	Compiled verified CSV assets (chart1–chart4), reconciled Stage 4 v1 vs. v2 methodology, and prepared final academic report.

3. TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional

The system was designed to satisfy six core functional requirements (FR-1 through FR-6 in Table 3.1): streaming and caching BeIR/msmarco passages with provenance manifests, segmenting passages into deterministic character chunks (300/50), generating 768-dimensional embeddings via local Ollama (nomic-embed-text), populating isolated FP32 (vector(768)) and FP16 (halfvec(768)) representation tables via in-database casting, building HNSW indexes (m = 16, ef_construction = 64), executing exact and HNSW similarity queries across controlled planner modes, and computing disaggregated storage, latency, Recall@K, pairwise agreement, and EXPLAIN (ANALYZE, BUFFERS) diagnostics.

3.1.2 Non-Functional

The three non-functional requirements (NFR-1 through NFR-3 in Table 3.1) focus on experimental reproducibility, measurement isolation, fail-closed database safety guards, automated testability (43/43 passing tests), and resource efficiency. Table 3.1 distinguishes between system design requirements and properties that were experimentally validated.

Table 3.1: Functional and non-functional requirements of the implemented benchmarking system.

Req. ID	Category	Requirement Description	Implementation & Validation Status
FR-1	Functional	Stream BeIR/msmarco passages into atomic local TSV caches with JSON metadata manifests.	Implemented in src/ingestion.py & src/scale_experiment.py; SHA-256 verified.
FR-2	Functional	Segment passages deterministically (chunk_size=300, overlap=50) with zero-overshoot slicing at 5k and 10k.	Implemented in src/preprocessing.py; verified exact 5,000 and 10,000 row counts.
FR-3	Functional	Generate 768-D embeddings locally via Ollama nomic-embed-text and cast to halfvec(768) in SQL.	Implemented in src/embeddings.py & SQL ::halfvec(768); 1:1 row parity verified.
FR-4	Functional	Build HNSW indexes (m=16, ef_construction=64) on FP32 (vector_cosine_ops) and FP16 (halfvec_cosine_ops).	Implemented in src/hnsw_experiment.py & src/scale_experiment.py; verified via pg_indexes.
FR-5	Functional	Execute Exact, Forced HNSW (ef_search=40), and Planner-Selected retrieval modes and capture Q0 EXPLAIN plans.	Validated across 6 configurations x 100 trials in Stage 4 v2 (chart2, chart4).
FR-6	Functional	Audit disaggregated storage (pg_relation_size, pg_toast, _fsm, _vm) and compute Recall@K & Agreement@K.	Validated in chart1_storage.csv, chart3_hnsw_recall.csv, chart3_pairwise_agreement.csv.
NFR-1	Non-Functional	Strict multi-database isolation so baseline and external databases cannot be mutated by scaling scripts.	Enforced by assert_test_database, assert_allowed_db, and assert_scale_mutation_db.
NFR-2	Non-Functional	Unbiased client latency measurement without connection setup, embedding inference, or GUC overhead inside timer.	Validated in Stage 4 v2 by pre-computing query vectors and moving SET GUCs outside timer.
NFR-3	Non-Functional	Automated regression testing across all pipeline modules.	Validated via 43/43 passing pytest tests in 10.27 s on compression_aware_rag_test.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The project is technically feasible on a standard local development workstation because PostgreSQL 18.4 and pgvector 0.8.4 provide native C-level support for both vector(768) and halfvec(768) data types, cosine distance operators (<=>), and HNSW indexing (vector_cosine_ops and

halfvec_cosine_ops) [2]. Local embedding generation is handled offline via Ollama serving nomic-embed-text [4], and the 10,000-chunk representation tables and HNSW indexes (79.20 MiB FP32; 35.35 MiB FP16) fit comfortably within local NVMe storage and PostgreSQL's 128 MB shared_buffers.

3.2.2 Economic Feasibility

No financial costing or monetary budget was required or measured for this project. Every software component—PostgreSQL, pgvector, Python, psycopg, pytest, Ollama, nomic-embed-text, and the Hugging Face BeIR/msmarco dataset—is open-source or publicly available at zero licensing cost, and all benchmarks executed on existing student laptop hardware without paid cloud database instances or commercial embedding API fees. From a qualitative resource perspective, reducing inclusive relation storage by 55.35%–55.37% and HNSW index size by 49.90%–49.98% allows a given memory and disk allocation to hold roughly twice as many indexed 768-dimensional vectors.

3.2.3 Social Feasibility

From an academic and engineering standpoint, the project promotes resource-conscious database design and reproducible evaluation. By demonstrating how coordinate-level half-precision storage avoids unnecessary out-of-line TOAST overhead and halves index memory footprints on local hardware, the work supports more efficient use of computing resources and provides a transparent, fully tested benchmark framework that other students and practitioners can inspect and reproduce.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

All experiments ran on a single local Windows 11 laptop whose hardware profile was verified from Windows Task Manager (Table 3.2): an **Intel Core Ultra 9 185H** processor, **32 GB installed RAM** (approximately 31.4 GB available to Windows), **Intel Arc integrated graphics**, an **NVMe SSD**, and an **Intel AI Boost NPU**. Neither the integrated GPU nor the NPU is claimed to have accelerated PostgreSQL query execution or Ollama embedding generation.

3.3.2 Software Specification

Table 3.2 lists the software and database parameters from the Stage 1–4 benchmark reports: **PostgreSQL 18.4**, **pgvector 0.8.4**, **Python 3.13.x**, psycopg 3.3.4, pytest 9.1.1, and local Ollama serving nomic-embed-text (768 dimensions), configured with shared_buffers = 128 MB, maintenance_work_mem = 64 MB, and work_mem = 4 MB. (Note: older repository summary files mention PostgreSQL 18.0, a pgvector 0.5.0 badge, and work_mem = 64 MB, report Python 3.13.2 vs. 3.13.14 in pytest headers, and disagree on the Ollama runtime version—listing 0.4.7 in the handoff vs. 0.32.13 in the Review 2 slides while the Stage 1–4 reports do not log an Ollama version string; we use the stage-specific PostgreSQL/pgvector settings as primary and leave the exact Ollama version as an unresolved source discrepancy.)

Table 3.2: Experimental hardware and software specifications.

Category	Component / Parameter	Verified Specification & Configuration Note
Hardware	Processor (CPU)	**Intel Core Ultra 9 185H** (verified from host Windows Task Manager)
Hardware	System Memory (RAM)	**32 GB installed RAM** (~31.4 GB available to Windows)

Category	Component / Parameter	Verified Specification & Configuration Note
Hardware	Graphics, Storage & NPU	Intel Arc integrated graphics; NVMe SSD; Intel AI Boost NPU present (no GPU/NPU acceleration claimed)
Software	Operating System	Windows 11 (win32 platform)
Software	Database & Vector Extension	**PostgreSQL 18.4** with **pgvector 0.8.4** (older README/handoff note 18.0 / 0.5.0 badge)
Software	Python & Libraries	**Python 3.13.x** (3.13.14 / 3.13.2), psycopg 3.3.4, datasets, requests, pytest 9.1.1
Software	Embedding Runtime	Local Ollama serving nomic-embed-text (768d; exact Ollama version unresolved across sources: handoff lists 0.4.7, Review 2 lists 0.32.13)
DB Config	Memory Parameters	shared_buffers = 128 MB; maintenance_work_mem = 64 MB; work_mem = 4 MB (handoff lists 64 MB)
DB Config	HNSW & Chunking	chunk_size = 300, overlap = 50; HNSW m = 16, ef_construction = 64, ef_search = 40

4. DESIGN APPROACH AND DETAILS

4.1 SYSTEM ARCHITECTURE

Figure 1 shows our implemented seven-stage system architecture. Passages from BeIR/msmarco [3] are streamed and saved atomically to local TSV caches with JSON metadata manifests (data/raw/*.meta.json), segmented into 300-character chunks with a 50-character overlap (src/preprocessing.py), and embedded into 768-dimensional float vectors via local Ollama nomic-embed-text (src/embeddings.py) [4]. To keep variable-length passage text from confounding vector storage and scan measurements, the database schema separates the parent documents and document_chunks ground-truth tables from dedicated two-column vector representation tables (chunk_id INT PRIMARY KEY, embedding vector(768) | halfvec(768)). HNSW indexes (m = 16, ef_construction = 64) are built on each representation table, and the evaluation harness collects disaggregated storage footprints, 100-trial retrieval latencies, Recall@K, pairwise agreement, and Q0 EXPLAIN (ANALYZE, BUFFERS) plans.

Figure 1: Implemented System Architecture and Compression-Aware Benchmarking Pipeline

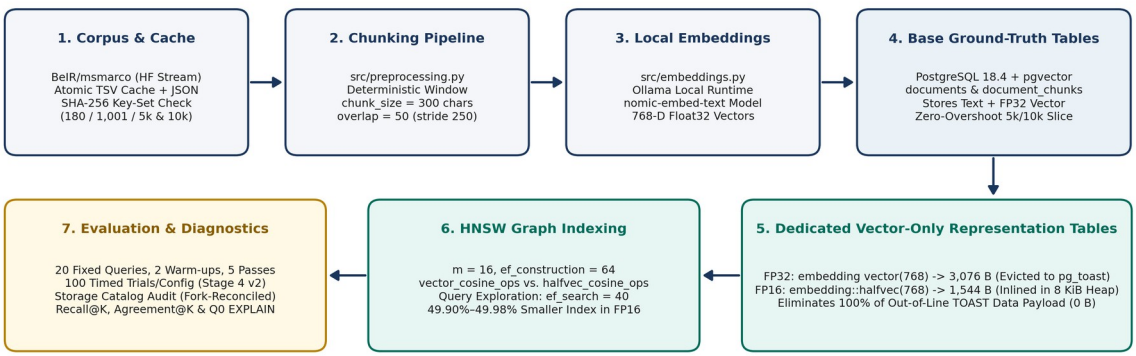


Figure 1: Implemented system architecture and compression-aware benchmarking pipeline.

To ensure strict experimental isolation across project stages, workloads are partitioned across five separate PostgreSQL databases guarded by fail-closed connection checks (Table 4.1). When Stage 4 reuses the first 1,001 embeddings from compression_aware_rag_exp, it opens the source connection in read-only mode (conn.read_only = True) after verifying 1,001 / 1,001 SHA-256 chunk-text hash matches.

Table 4.1: Multi-database layout and isolation policy across project stages.

Database Name	Project Stage & Role	Corpus Size	Tables & Indexes Stored	Isolation Policy
compression_aware_rag	Review 2 Baseline	100 docs / 180 chunks	Monolithic documents + document_chunks + HNSW	Read-only baseline; zero modifications in Stages 1–4
smart_document_chatbot	External Course DB	3 public tables	Separate application tables	Untouched; excluded by connection guards
compression_aware_rag_exp	Stages 1–3 Exp DB	550 docs / 1,001 chunks	Base tables + FP32/FP16 tables + HNSW indexes	Used in Stages 1–3; read-only during Stage 4 reuse
compression_aware_rag_scale	Stage 4 Scaling DB	5,457 docs / 10,000 chunks	Base tables + 5k & 10k fp32/halfvec tables + 4 HNSW idx	Exclusive target for Stage 4 scaling benchmarks
compression_aware_rag_test	Pytest Test DB	Ephemeral test data	Created and truncated per test via conftest.py	Enforced by autouse test-database guard

4.2 DESIGN

4.2.1 Data Flow Diagram

Figure 2 presents the Level-1 Data Flow Diagram (DFD) of the implemented system. Process **P1** streams raw BeIR/msmarco records, applies deterministic sliding-window chunking (300/50), validates SHA-256 key-set parity against the 1,001-chunk baseline, and writes atomic TSV/JSON caches (**D1**). Process **P2** reuses the 1,001 validated embeddings from compression_aware_rag_exp, generates 768-dimensional nomic-embed-text vectors for new chunks via Ollama, and populates the base PostgreSQL tables (**D2**). Process **P3** populates the 5,000- and 10,000-chunk FP32 and FP16 representation tables via in-database casting (embedding::halfvec(768)) and builds the four HNSW indexes (**D3**). Finally, Process **P4** executes the 100-trial retrieval benchmark and Q0 EXPLAIN (ANALYZE, BUFFERS) diagnostics over the 20 pre-embedded evaluation queries, while Process **P5** computes fork-reconciled storage metrics, Recall@K, and pairwise Agreement@K to produce the benchmark reports and CSV artifacts (**D4**).

Figure 2: Level-1 Data Flow Diagram (DFD) of the Implemented Vector Indexing and Benchmarking System

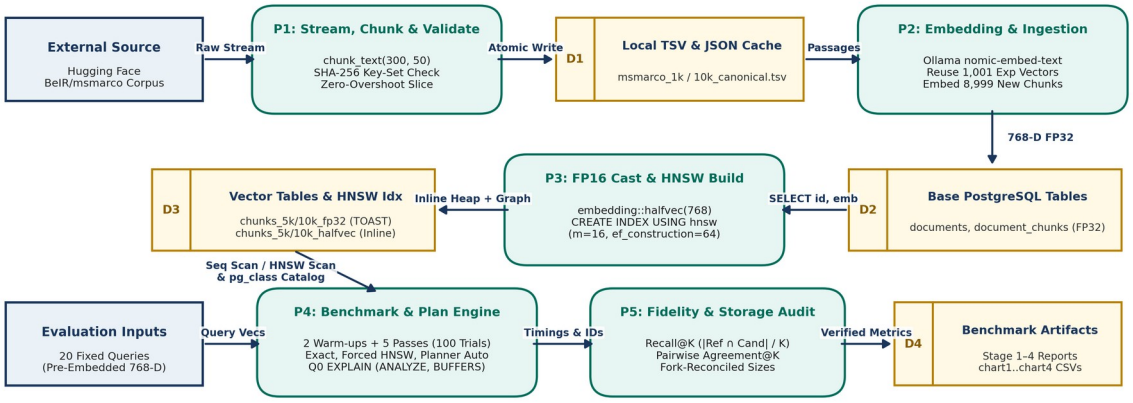


Figure 2: Level-1 Data Flow Diagram (DFD) of the implemented vector indexing and benchmarking system.

4.2.2 Class Diagram

Inspection of the project source code (src/*.py) shows that the Python implementation is organized around modular procedural functions rather than custom object-oriented Python classes. Rather than inventing fictional wrapper classes, Figure 3 presents the verified UML Database Entity-Relationship and Procedural Module Diagram. The left and center panels show the relational table entities (documents, document_chunks, chunks_{1k|5k|10k}_fp32, and chunks_{1k|5k|10k}_halfvec), primary/foreign keys (ON DELETE CASCADE), HNSW index definitions, and physical storage placement (out-of-line pg_toast vs. inline 8 KiB heap), while the right panels show the procedural Python modules that operate on those entities.

Figure 3: UML Database Entity-Relationship and Procedural Module Diagram (Verified Implementation)

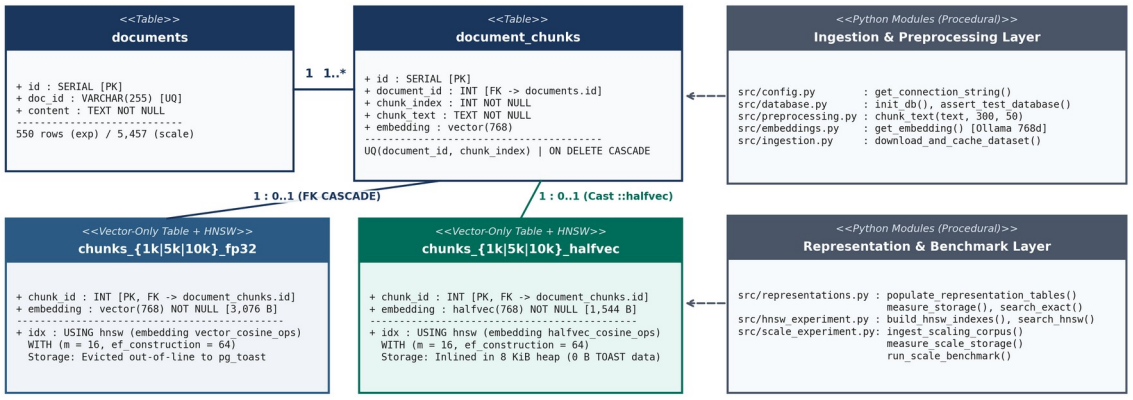


Figure 3: UML database entity-relationship and procedural module diagram (verified implementation).

5. METHODOLOGY AND TESTING

5.1 MODULE DESCRIPTION

Table 5.1 summarizes the responsibilities of the implemented Python modules in src/. Across all experimental stages, passages from BeIR/msmarco [3] are segmented using chunk_size = 300 characters and overlap = 50 characters (250-character stride), embedded into 768-dimensional vectors using local Ollama nomic-embed-text [4], and indexed with HNSW graph parameters m = 16, ef_construction = 64, and query exploration ef_search = 40 [2, 6].

Table 5.1: Functional responsibilities of the implemented Python modules in src/.

Module File	Primary Responsibility in the Pipeline
src/config.py	Loads environment variables and builds PostgreSQL connection strings and Ollama API URLs.
src/database.py	Initializes the base documents and document_chunks schema, registers pgvector, and enforces the test-database guard.
src/preprocessing.py	Implements chunk_text(text, chunk_size=300, overlap=50) for deterministic sliding-window text segmentation.
src/embeddings.py	Queries the local Ollama nomic-embed-text HTTP endpoint and validates that returned vectors have length 768.
src/ingestion.py	Streams BeIR/msmarco from Hugging Face, writes atomic TSV caches and JSON manifests, and ingests batches into PostgreSQL.
src/search.py & src/benchmark.py	Implements the Review 2 baseline exact and HNSW search queries and the 180-chunk baseline benchmark runner.
src/representations.py	Manages Stage 2 chunks_embedding_fp32 and chunks_embedding_halfvec tables, storage catalog queries, and 20-query exact benchmarks.
src/hnsw_experiment.py	Builds Stage 3 HNSW indexes at 1,001 chunks, verifies index scan plans via EXPLAIN, and runs Forced vs. Planner-Selected benchmarks.
src/scale_experiment.py	Executes Stage 4 10k cache validation, zero-overshoot 5k/10k ingestion, fork-reconciled storage audits, and the 6-configuration Stage 4 v2 benchmark.

5.2 TESTING

Table 5.2 documents our automated pytest test suites alongside the experimental timing boundaries used at each stage. For retrieval benchmarks in Stages 2, 3, and 4, we evaluated the same 20 domain-diverse queries (EVALUATION_QUERIES), pre-computed all 20 query embeddings before starting the timing loop, executed **2 unmeasured warm-up passes** to warm shared_buffers, and recorded **5 measured passes (100 timed trials per configuration)** while alternating execution order (FP32 -> FP16 on even passes, FP16 -> FP32 on odd passes) over a single persistent psycopg connection. Crucially, Stage 3 (1,001 chunks) timed a session transaction/GUC block per trial (BEGIN; SET LOCAL ...; SELECT ...; COMMIT;), whereas Stage 4 v2 (5,000 and 10,000 chunks) applied session GUC settings once outside the timed loop and timed strictly cur.execute() and cur.fetchall().

Statistically, the 100 timed trials per configuration in Stages 2–4 are repeated executions of the same 20 fixed queries (20 queries × 5 measured passes), not 100 independent queries from a general population. Reported mean, median, IQR, minimum, maximum, and standard deviation describe the observed distribution of these repeated runs; no confidence intervals, significance tests, or population-level claims were established, and alternating FP32/FP16 order reduces systematic cache-order bias without eliminating all warm-cache or OS scheduling effects. Although stage-report latency tables omit a K header (and K is not assumed from Review 2's K = 3 or sampled EXPLAIN plans), src/representations.py, src/hnsw_experiment.py, and run_scale_benchmark() (src/scale_experiment.py) pass top_k = max(top_k_values) = 10 (LIMIT 10) in timed loops, distinct from K in {3, 5, 10} fidelity runs.

Table 5.2: Automated pytest verification suites and experimental timing boundaries across stages.

Milestone / Stage	Target Database	Scale	Trial Protocol & Timing Boundary	Pytest Suite
Review 2 Prototype	compression_aware_rag	180 chunks	5 queries x 5 runs (K=3); preliminary pre-stabilization run	5 / 5 passed
Review 2 Baseline	compression_aware_rag	180 chunks	5 queries x 5 runs (25 trials, K=3); stabilized connection & plan check	11 / 11 passed
Stage 1 (Ingestion)	compression_aware_rag_exp	1,001 chunks	Atomic HF streaming cache & manifest validation (550 docs)	19 / 19 passed
Stage 2 (FP16 Exact)	compression_aware_rag_exp	1,001 chunks	20 queries x 5 runs (100 trials/rep); dedicated tables, unindexed Seq Scan	27 / 27 passed
Stage 3 (1k HNSW)	compression_aware_rag_exp	1,001 chunks	20 queries x 5 runs (100 trials/config); timed per-trial session Tx/GUC block	34 / 34 passed
Stage 4 v2 (5k & 10k)	compression_aware_rag_scale	5,000 & 10,000	20 queries x 5 runs (100 trials x 6 configs); GUCs outside timer (exec+fetch only)	**43 / 43 passed**

6. PROJECT DEMONSTRATION

6.1 REPRESENTATION TABLE DEFINITIONS AND HALF-PRECISION CASTING

To guarantee that FP32 and FP16 tables contain identical underlying embeddings, populate_scale_representation_tables() in src/scale_experiment.py copies the first N rows (5,000 or 10,000) from document_chunks and casts the stored 32-bit vector to halfvec(768) directly inside PostgreSQL (Listing 6.1). Afterward, validate_scale_representations() verifies row counts, zero NULLs, 768 dimensions (vector_dims), and zero mismatched IDs via a FULL OUTER JOIN.

Listing 6.1: Dedicated Vector Representation Tables and In-Database FP16 Casting (src/scale_experiment.py)

```
CREATE TABLE IF NOT EXISTS chunks_10k_fp32 (
    chunk_id INT PRIMARY KEY REFERENCES document_chunks(id) ON DELETE
CASCADE,
    embedding vector(768) NOT NULL
);
CREATE TABLE IF NOT EXISTS chunks_10k_halfvec (
    chunk_id INT PRIMARY KEY REFERENCES document_chunks(id) ON DELETE
CASCADE,
    embedding halfvec(768) NOT NULL
);

-- Deterministic in-database conversion from FP32 to FP16 (halfvec)
INSERT INTO chunks_10k_fp32 (chunk_id, embedding)
SELECT id, embedding FROM document_chunks ORDER BY id ASC LIMIT 10000;

INSERT INTO chunks_10k_halfvec (chunk_id, embedding)
SELECT id, embedding::halfvec(768) FROM document_chunks ORDER BY id ASC LIMIT
10000;
```

6.2 EXACT-SEARCH AND HNSW RETRIEVAL QUERY PATTERNS

All similarity queries compute cosine distance (<=>), defined as one minus cosine similarity between 768-dimensional vectors u and v:

$$d_cos(u, v) = 1 - (sum_\{i=1\}\^\{768\} u_i v_i) / (||u||_2 * ||v||_2) \quad \textbf{(6.1)}$$

Listing 6.2 shows the canonical SQL query patterns verified in Stage 4 v2. In exact sequential search (search_scale_exact), ordering by the projected column alias ORDER BY distance ASC evaluates the distance expression once per row; repeating ORDER BY embedding <=> %s in Stage 4 v1 caused PostgreSQL to evaluate the expression and detoast FP32 vectors twice per row. Conversely, for HNSW queries (search_scale_hnsw), PostgreSQL's planner requires the explicit operator expression ORDER BY embedding <=> %s::{cast_type} ASC in order to match the index's operator class (vector_cosine_ops or halfvec_cosine_ops).

Listing 6.2: Canonical Exact-Search vs. HNSW Index-Search SQL Patterns (src/scale_experiment.py)

```
-- 1. HNSW Index Creation (m = 16, ef_construction = 64)
CREATE INDEX chunks_10k_halfvec_hnsw_idx ON chunks_10k_halfvec
    USING hnsw (embedding halfvec_cosine_ops) WITH (m = 16, ef_construction =
64);

-- 2. Exact Sequential Search (orders by 'distance' alias to avoid double
detoasting)
SET enable_indexscan = off; SET enable_bitmapscan = off; SET enable_seqscan =
on;
SELECT chunk_id, (embedding <=> %s::halfvec(768)) AS distance
FROM chunks_10k_halfvec ORDER BY distance ASC LIMIT 10;

-- 3. Forced HNSW Search (retains direct <=> operator in ORDER BY for index
matching)
SET enable_indexscan = on; SET enable_bitmapscan = on; SET enable_seqscan =
off; SET hnsw.ef_search = 40;
SELECT chunk_id, (embedding <=> %s::halfvec(768)) AS distance
FROM chunks_10k_halfvec ORDER BY embedding <=> %s::halfvec(768) ASC LIMIT 10;
```

6.3 BENCHMARK TIMING AND QUERY-PLAN VERIFICATION

Listing 6.3 demonstrates how run_scale_benchmark() in src/scale_experiment.py applies session GUC settings once per pass outside the timed loop so that time.perf_counter() measures strictly cur.execute() and cur.fetchall().

Listing 6.3: Stage 4 v2 Timed Query Execution Loop (src/scale_experiment.py)

```
# Session GUC configuration is applied once per pass OUTSIDE the timed loop:
with conn.cursor() as cur:
    cur.execute(f"SET enable_seqscan = {'on' if enable_seqscan else 'off'};")
    cur.execute(f"SET hnsw.ef_search = {int(ef_search)};")
for q_idx in range(len(EVALUATION_QUERIES)):
    q_emb = query_embeddings[q_idx]
    start_t = time.perf_counter()
    _ = search_scale_hnsw(conn, scale, rep, q_emb, top_k=10, ef_search=40,
                          enable_seqscan=enable_seqscan, apply_guc=False)
    lat_ms = (time.perf_counter() - start_t) * 1000.0
```

7. RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE)

7.1 PHYSICAL STORAGE, TOAST BEHAVIOUR, AND HNSW BUILD-TIME OBSERVATIONS

At the datum level, pg_column_size(embedding) across Stages 2 and 4 returned **3,076 bytes** (min = max = avg) for vector(768) and **1,544 bytes** for halfvec(768)—a **49.80% empirical datum reduction (1.992x)** (Table 7.1). Whereas 1,544 bytes matches the documented pgvector halfvec formula (2d + 8 = 1,544 B), 3,076 bytes is 4 bytes smaller than the documented vector formula (4d + 8 = 3,080 B, or 49.87% reduction). Both figures are preserved transparently.

Table 7.1: Coordinate datum size comparison for 768-dimensional vectors.

Datum Metric	FP32 (vector(768))	FP16 (halfvec(768))	Reduction (Ratio)	Storage Layout Impact
Raw Coordinate Bytes	3,072 B (768 x 4 B)	1,536 B (768 x 2 B)	50.00% lower (2.000x)	IEEE 754 float coordinate array
Documented Formula Size	3,080 B (4d + 8 B)	1,544 B (2d + 8 B)	49.87% lower (1.995x)	Documented varlena + dim header
Empirical pg_column_size()	**3,076 B**	**1,544 B**	**49.80% lower (1.992x)**	4-byte empirical vs. formula difference
Placement vs. ~2 KiB Limit	Evicted to pg_toast	Inlined in main heap	100% TOAST data eliminated	Empty 8 KiB TOAST index shell remains

Table 7.2 and Figure 4 present the disaggregated storage footprints across 1,001, 5,000, and 10,000 chunks (chart1_storage.csv, where 1 MiB = 1,048,576 B and 1 KiB = 1,024 B), keeping three storage scopes distinct:

$$Total_Inclusive = Heap_main + TOAST_Data_main + TOAST_Idx_main + PK_Idx_main + HNSW_Idx_main + Aux_Forks_(FSM+VM) \quad (7.1)$$

- **Pre-HNSW Base Relation Storage:** In the evaluated vector-only schema, halfvec(768) reduced out-of-line TOAST data payloads to **0 bytes** at every scale (vs. 3.91 MiB, 19.53 MiB, and 39.06 MiB for FP32), while an empty **8 KiB (8,192 B) TOAST index shell** remained allocated. Base

relation size decreased by **60.11%** at 1,001 chunks (4.13 MiB to 1.65 MiB), **60.52%** at 5,000 chunks (20.20 MiB to 7.98 MiB), and **60.56%** at 10,000 chunks (40.31 MiB to 15.90 MiB).

- **Standalone HNSW Index Footprint & Build Times:** HNSW index size (pg_relation_size) decreased by **49.90%** at 1,001 chunks (3.90 MiB to 1.95 MiB) and by **49.98%** at both 5,000 chunks (19.48 MiB to 9.74 MiB) and 10,000 chunks (38.89 MiB to 19.45 MiB). Single-run wall-clock HNSW construction times (0.2334 s vs. 0.1606 s at 1k; 1.4615 s vs. 1.2640 s at 5k; 3.7617 s vs. 2.6411 s at 10k) were lower for FP16 at each tested size, though these are single-run measurements rather than repeated-trial distributions.
- **Post-Index Inclusive Relation Storage:** Inclusive size (pg_total_relation_size, reconciled to the exact byte via _fsm and _vm auxiliary forks) decreased by **55.35%** at 5,000 chunks (39.68 MiB to 17.72 MiB) and **55.37%** at 10,000 chunks (79.20 MiB to 35.35 MiB). No single post-index pg_total_relation_size query was recorded at 1,001 chunks in Stage 3, so that cell is left unrecorded.

Table 7.2: Disaggregated storage footprint and single-run HNSW build durations across 1,001, 5,000, and 10,000 chunks.

Storage Component	1,001 FP32	1,001 FP16	5,000 FP32	5,000 FP16	10,000 FP32	10,000 FP16
Table Heap ('main')	57,344 B (56.0 KiB)	1,646,592 B (1.57 MiB)	262,144 B (256.0 KiB)	8,192,000 B (7.81 MiB)	524,288 B (512.0 KiB)	16,384,000 B (15.63 MiB)
TOAST Data Table ('main')	4,104,192 B (3.91 MiB)	**0 B (0 MiB)**	20,480,000 B (19.53 MiB)	**0 B (0 MiB)**	40,960,000 B (39.06 MiB)	**0 B (0 MiB)**
TOAST B-Tree Index ('main')	65,536 B (64.0 KiB)	8,192 B (8.0 KiB)	245,760 B (240.0 KiB)	8,192 B (8.0 KiB)	466,944 B (456.0 KiB)	8,192 B (8.0 KiB)
Primary Key Index ('main')	40,960 B (40.0 KiB)	40,960 B (40.0 KiB)	131,072 B (128.0 KiB)	131,072 B (128.0 KiB)	245,760 B (240.0 KiB)	245,760 B (240.0 KiB)
Auxiliary Forks (_fsm + _vm)	65,536 B (64.0 KiB)	32,768 B (32.0 KiB)	65,536 B (64.0 KiB)	32,768 B (32.0 KiB)	73,728 B (72.0 KiB)	32,768 B (32.0 KiB)
Pre-HNSW Base Relation Total	**4,333,568 B (4.13 MiB)**	**1,728,512 B (1.65 MiB)**	**21,184,512 B (20.20 MiB)**	**8,364,032 B (7.98 MiB)**	**42,270,720 B (40.31 MiB)**	**16,670,720 B (15.90 MiB)**
Base Relation Reduction	Baseline	**60.11% lower**	Baseline	**60.52% lower**	Baseline	**60.56% lower**
HNSW Index Size ('main')	**4,087,808 B (3.90 MiB)**	**2,048,000 B (1.95 MiB)**	**20,422,656 B (19.48 MiB)**	**10,215,424 B (9.74 MiB)**	**40,779,776 B (38.89 MiB)**	**20,398,080 B (19.45 MiB)**
HNSW Index Reduction	Baseline	**49.90% lower**	Baseline	**49.98% lower**	Baseline	**49.98% lower**
Post-Index Inclusive Total	Not recorded (Stage 3)	Not recorded (Stage 3)	**41,607,168 B (39.68 MiB)**	**18,579,456 B (17.72 MiB)**	**83,050,496 B (79.20 MiB)**	**37,068,800 B (35.35 MiB)**
Inclusive Total Reduction	—	Not recorded	Baseline	**55.35% lower**	Baseline	**55.37% lower**

Storage Component	1,001 FP32	1,001 FP16	5,000 FP32	5,000 FP16	10,000 FP32	10,000 FP16
Single-Run HNSW Build Time	0.2334 s	0.1606 s (31.18% lower)	1.4615 s	1.2640 s (13.52% lower)	3.7617 s	2.6411 s (29.79% lower)

Figure 4: Physical Storage Footprint Across Corpus Scales (chart1_storage.csv)

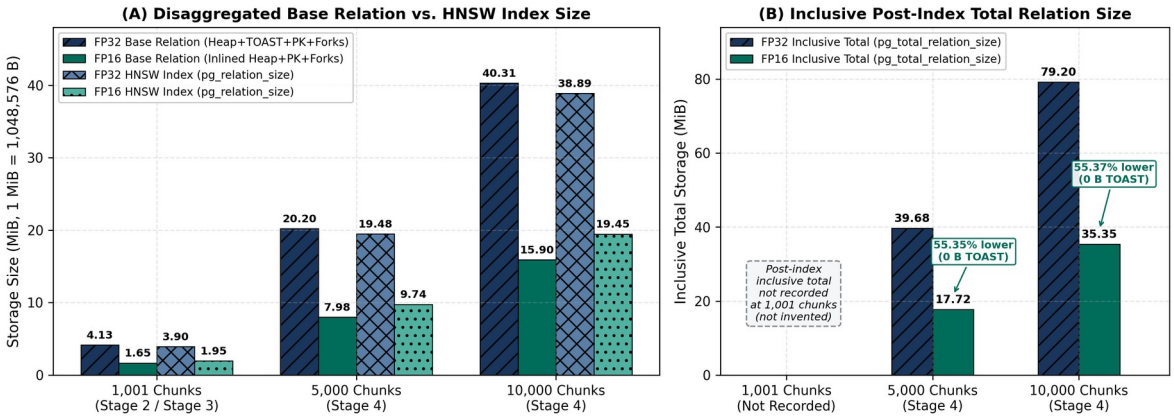


Figure 4: Physical storage footprint comparison across 1,001, 5,000, and 10,000 chunks (chart1_storage.csv).

7.2 EXACT-SEARCH AND FORCED-HNSW LATENCY COMPARISONS

In our earlier milestones, the stabilized 180-chunk baseline recorded 2.180 ms mean (2.049 ms median) for exact sequential search and 2.270 ms mean (2.125 ms median) for forced HNSW search. At 1,001 chunks on the dedicated representation tables, Stage 2 unindexed exact search averaged 3.320 ms (3.120 ms median) for FP32 and 1.784 ms (1.644 ms median) for FP16 (46.27% lower mean, 1.86x speedup). In Stage 3 (1,001 chunks, where the timer included per-trial session transaction/GUC setup), Forced HNSW averaged 3.165 ms (2.866 ms median) for FP32 and 2.984 ms (2.788 ms median) for FP16.

Table 7.3 and Figure 5 present the Stage 4 v2 latency results across 5,000 and 10,000 chunks (chart4_latency.csv; 100 timed trials per configuration with GUC statements outside the timer). In exhaustive exact search (Seq Scan), the inline FP16 representation recorded an **83.91% lower mean latency (6.22x speedup)** and median latency by **85.96% (7.12x speedup)** at 5,000 chunks, and reduced mean latency by **86.18% (7.24x speedup)** and median latency by **87.55% (8.03x speedup)** at 10,000 chunks, winning **100 out of 100 paired trials** at both scales. In contrast, under Forced HNSW (ef_search = 40), both representations completed queries in roughly 3.37–4.13 ms on the mean (2.92–3.82 ms median), with FP16 slightly faster at 5,000 chunks (3.372 ms vs. 4.129 ms mean) and slightly slower at 10,000 chunks (3.856 ms vs. 3.674 ms mean, or 4.95% higher mean and 11.86% higher median). Because both indexes fit inside PostgreSQL's 128 MB shared_buffers and HNSW visits only a small candidate subgraph, coordinate compression does not produce a large HNSW speedup under warm-cache conditions.

Table 7.3: Stage 4 v2 client-observed retrieval latency across all six configurations at 5,000 and 10,000 chunks.

Scale & Config	Mode & Precision	Mean (ms)	Median (ms)	IQR (ms)	Min (ms)	Max (ms)	StDev (ms)	Relative Change / Speedup vs. FP32
5k (Config 1)	FP32 Exact (Seq Scan)	**42.285**	**44.694**	16.018	10.907	57.104	10.083	Baseline (1.00x)
5k (Config 2)	FP16 Exact (Seq Scan)	**6.802**	**6.277**	2.606	3.203	15.669	2.318	**83.91% lower mean (6.22x) / 85.96% lower med (7.12x)**
5k (Config 3)	FP32 Forced HNSW	**4.129**	**3.371**	1.641	2.449	42.670	4.073	Baseline (1.00x)
5k (Config 4)	FP16 Forced HNSW	**3.372**	**2.923**	1.571	2.386	6.501	0.952	18.32% lower mean (1.22x) / 13.28% lower med (1.15x)
5k (Config 5)	FP32 Planner Auto (Idx)	3.765	3.504	1.389	2.457	10.949	1.372	Baseline (1.00x)
5k (Config 6)	FP16 Planner Auto (Idx)	3.568	3.293	1.681	2.358	6.427	1.062	5.23% lower mean (1.06x) / 6.02% lower med (1.06x)
10k (Config 1)	FP32 Exact (Seq Scan)	**89.004**	**92.664**	32.376	50.664	125.432	20.031	Baseline (1.00x)
10k (Config 2)	FP16 Exact (Seq Scan)	**12.301**	**11.541**	2.866	7.205	22.226	3.163	**86.18% lower mean (7.24x) / 87.55% lower med (8.03x)**
10k (Config 3)	FP32 Forced HNSW	**3.674**	**3.415**	1.548	2.420	6.420	0.990	Baseline (1.00x)
10k (Config 4)	FP16 Forced HNSW	**3.856**	**3.820**	1.796	2.482	7.740	1.092	4.95% higher mean (0.95x) / 11.86% higher med (0.89x)
10k (Config 5)	FP32 Planner Auto (Idx)	3.678	3.482	1.593	2.506	6.618	1.015	Baseline (1.00x)
10k (Config 6)	FP16 Planner Auto (Idx)	3.724	3.566	1.759	2.342	7.162	1.101	1.25% higher mean (0.99x) / 2.41% higher med (0.98x)

Figure 5: Client-Observed Retrieval Latency Under Stage 4 v2 Harness (chart4_latency.csv)

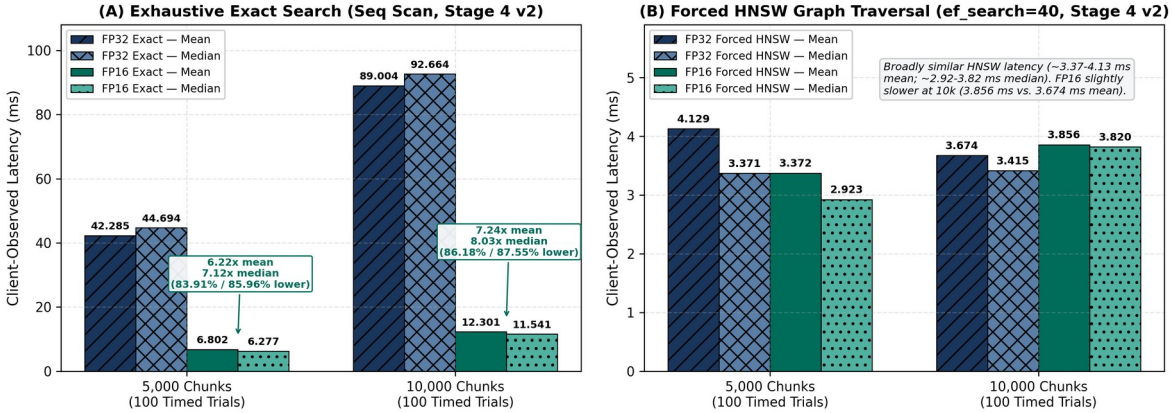


Figure 5: Client-observed retrieval latency under the Stage 4 v2 benchmark harness (chart4_latency.csv).

7.3 RETRIEVAL FIDELITY: RECALL@K AND PAIRWISE CANDIDATE-SET AGREEMENT

Following standard ANN evaluation practice of pairing latency measurements with exact-ground-truth recall [9], we evaluate retrieval quality using two separate set-overlap metrics averaged across the 20 evaluation queries (K in {3, 5, 10}). First, representation-specific Recall@K (chart3_hnsw_recall.csv) compares a method's top-K chunk IDs Top-K_Cand(q) against the uncompressed FP32 exact sequential scan ground truth Top-K_FP32_Exact(q) at the same scale:

$$Recall@K = (1 / 20) * sum_{q=1}^{20} (| Top\text{-}K_FP32_Exact(q) \ intersect \ Top\text{-}K_Cand(q) | / K) \quad (7.2)$$

Second, pairwise candidate-set agreement (chart3_pairwise_agreement.csv) measures how closely the top-K candidates returned by FP32 HNSW (S_FP32(q)) match those returned by FP16 HNSW (S_FP16(q)) at ef_search = 40:

$$Agreement@K = (1 / 20) * sum_{q=1}^{20} (| S_FP32(q) \ intersect \ S_FP16(q) | / K) \quad (7.3)$$

Equation (7.3) divides the intersection size by K (len(ref_set.intersection(cand_set)) / float(top_k) in src/scale_experiment.py), so it is a symmetric top-K overlap fraction rather than Jaccard similarity, and it does not measure semantic relevance, LLM generation quality, or internal HNSW graph connectivity.

Table 7.4 and Figure 6 (plotted on a zero-based 0.0–1.0 axis) show that unindexed FP16 exact search achieved 0.9833–1.0000 overlap with FP32 exact search across all scales (the historical Stage 2 Recall@3 = 0.9833 at 1,001 chunks equals 59/60 under Equation (7.2)'s unordered set-intersection formula; because set overlap does not measure within-set rank order, the Stage 2 report's wording about a 'minor order perturbation' is not independently established by the aggregate figure). Under HNSW (ef_search = 40), FP16 achieved **0.9600 (96.0%) Recall@10** at both 5,000 and 10,000 chunks. Notice that the 10,000-chunk FP32 HNSW sequence decreases slightly across K (0.9667 at K=3, 0.9500 at K=5, 0.9450 at K=10), and the 5,000-chunk values dip slightly from K=3 (0.9167) to K=5 (0.9100). Unlike cumulative information-retrieval recall computed against a fixed set of relevant documents, Equation (7.2) evaluates separate LIMIT K queries where both the reference set and the denominator K change with each cutoff; under this definition, overlap fractions can decrease as K increases without indicating a benchmark error, and the aggregate figures do not establish how misses are distributed across specific rank bands. Table 7.5 and Figure 7 show that pairwise top-10

agreement between FP32 HNSW and FP16 HNSW reached **0.9850** at 5,000 chunks and **0.9500** at 10,000 chunks.

Table 7.4: Representation-specific Recall@K against uncompressed FP32 exact sequential scan ground truth.

Corpus Scale	Evaluated Configuration	Recall@3	Recall@5	Recall@10	Ground-Truth Reference	Source Artifact
180 Chunks (Baseline)	FP32 Forced HNSW (ef_s=40)	1.0000	—	1.0000*	180c FP32 Exact Seq Scan	baseline_benchmark.md (*10q)
1,001 Chunks (Stage 2)	FP16 Exact Search (Seq Scan)	0.9833	1.0000	1.0000	1,001c FP32 Exact Seq Scan	stage2_fp16_evaluation.md
1,001 Chunks (Stage 3)	FP32 & FP16 HNSW (ef_s=40)	1.0000	1.0000	1.0000	1,001c FP32 Exact Seq Scan	stage3_hnsw_benchmark.md
5,000 Chunks (Stage 4)	FP16 Exact Search (Seq Scan)	1.0000	0.9900	0.9950	5,000c FP32 Exact Seq Scan	stage4_scale_benchmark.md
5,000 Chunks (Stage 4)	**FP32 HNSW (vector, ef_s=40)**	**0.9167**	**0.9100**	**0.9550**	5,000c FP32 Exact Seq Scan	chart3_hnsw_recall.csv
5,000 Chunks (Stage 4)	**FP16 HNSW (halfvec, ef_s=40)**	**0.9167**	**0.9100**	**0.9600**	5,000c FP32 Exact Seq Scan	chart3_hnsw_recall.csv
10,000 Chunks (Stage 4)	FP16 Exact Search (Seq Scan)	1.0000	0.9900	1.0000	10,000c FP32 Exact Seq Scan	stage4_scale_benchmark.md
10,000 Chunks (Stage 4)	**FP32 HNSW (vector, ef_s=40)**	**0.9667**	**0.9500**	**0.9450**	10,000c FP32 Exact Seq Scan	chart3_hnsw_recall.csv
10,000 Chunks (Stage 4)	**FP16 HNSW (halfvec, ef_s=40)**	**0.9500**	**0.9400**	**0.9600**	10,000c FP32 Exact Seq Scan	chart3_hnsw_recall.csv

Figure 6: Representation-Specific ANN Recall@K vs. Exhaustive FP32 Ground Truth (Zero-Based 0–1 Axis)

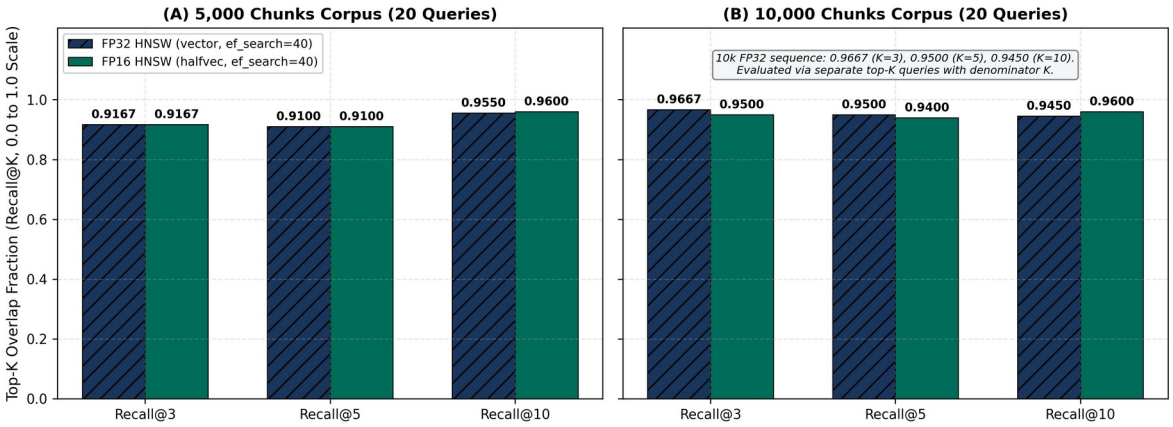


Figure 6: Representation-specific ANN Recall@K on a zero-based 0.0–1.0 vertical axis (chart3_hnsw_recall.csv).

Table 7.5: Pairwise top-K candidate-set overlap fraction between FP32 HNSW and FP16 HNSW at ef_search = 40.

Corpus Scale	Comparison Pair	Agreement @3	Agreement @5	Agreement @10	Metric Formula & Source
1,001 Chunks (Stage 3)	FP32 HNSW vs. FP16 HNSW	1.0000 (100.00%)	1.0000 (100.00%)	1.0000 (100.00%)	Mean |S_FP32(q) ∩ S_FP16(q)| / K (stage3_hnsw_benchmark.md)
5,000 Chunks (Stage 4)	**FP32 HNSW vs. FP16 HNSW**	**0.9667 (96.67%)**	**0.9800 (98.00%)**	**0.9850 (98.50%)**	Mean |S_FP32(q) ∩ S_FP16(q)| / K (chart3_pairwise_agreement.csv)
10,000 Chunks (Stage 4)	**FP32 HNSW vs. FP16 HNSW**	**0.9667 (96.67%)**	**0.9600 (96.00%)**	**0.9500 (95.00%)**	Mean |S_FP32(q) ∩ S_FP16(q)| / K (chart3_pairwise_agreement.csv)

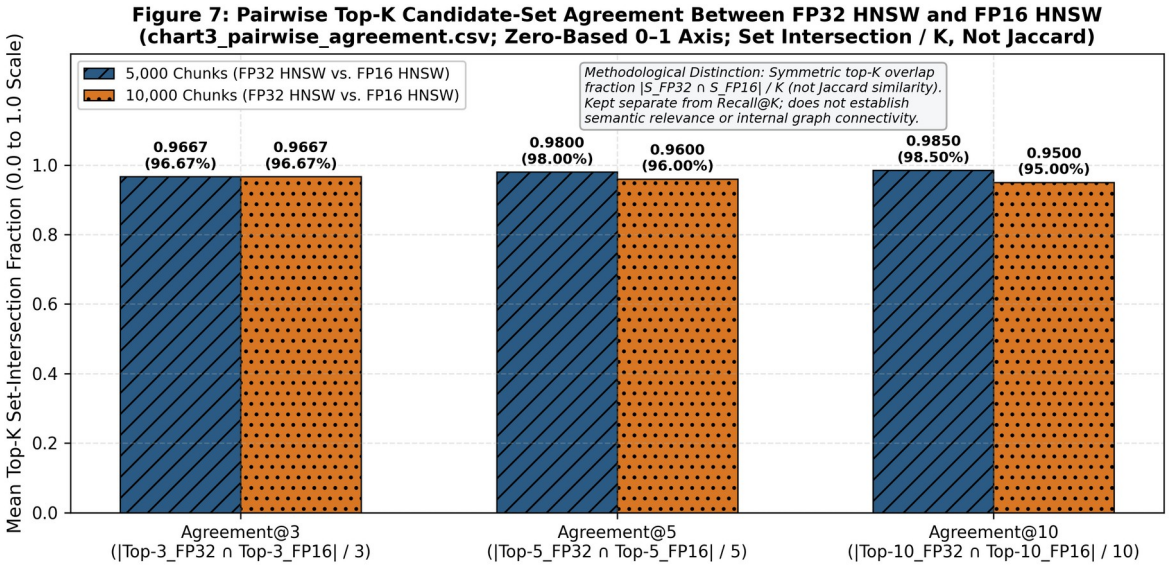


Figure 7: Pairwise top-K candidate-set agreement between FP32 HNSW and FP16 HNSW (chart3_pairwise_agreement.csv).

7.4 QUERY-PLAN, SHARED-BUFFER DIAGNOSTICS, AND PLANNER BEHAVIOUR

Table 7.6 and Figure 8 report the single-query Query 0 (Q0: 'what was the purpose of the manhattan project') diagnostic plans from EXPLAIN (ANALYZE, BUFFERS) (chart2_q0_diagnostics.csv). In exact sequential scans, FP16 required **93.32% fewer buffer hits** at 1,001 chunks (3,010 to 201) and **93.35% fewer buffer hits** at both 5,000 chunks (15,032 to 1,000) and 10,000 chunks (30,064 to 2,000). In forced HNSW scans, FP16 required **13.61% fewer buffer hits** at 1,001 chunks, **9.28% fewer** at 5,000 chunks, and **8.92% fewer** at 10,000 chunks. Because Q0 diagnostics come from a single query execution (shared read = 0 in those recorded plans) and shared hit counts runtime buffer accesses (including repeated touches to the same page), these buffer reductions are **consistent with**—rather than standalone proof of—the 100-trial exact-search latency speedups.

Table 7.6: Single-query Query 0 (Q0) EXPLAIN (ANALYZE, BUFFERS) diagnostic observations.

Stage & Scale	Retrieval Mode	Vector Type	Scan Node	Q0 Shared-Buffer Hits (read=0)	Q0 Limit Time (ms)	Q0 Server Exec Time (ms)	Buffer Hit Reduction vs. FP32
Stage 2 (1,001c)	Exact Search	FP32 (vector)	Seq Scan	3,010 hits	1.795 ms	1.801 ms	Baseline
Stage 2 (1,001c)	Exact Search	FP16 (halfvec)	Seq Scan	201 hits	0.398 ms	0.404 ms	**93.32% fewer buffer hits (14.98x)**
Stage 3 (1,001c)	Forced HNSW	FP32 (vector)	Index Scan	595 hits	—	1.267 ms	Baseline
Stage 3 (1,001c)	Forced HNSW	FP16 (halfvec)	Index Scan	514 hits	—	0.886 ms	**13.61% fewer buffer hits**
Stage 4 (5,000c)	**Exact Search**	**FP32 (vector)**	**Seq Scan**	**15,032 hits**	**24.527 ms**	**24.540 ms**	Baseline
Stage 4 (5,000c)	**Exact Search**	**FP16 (halfvec)**	**Seq Scan**	**1,000 hits**	**2.980 ms**	**2.995 ms**	**93.35% fewer buffer hits (15.03x)**
Stage 4 (5,000c)	**Forced HNSW**	**FP32 (vector)**	**Index Scan**	**830 hits**	**0.560 ms**	**0.571 ms**	Baseline
Stage 4 (5,000c)	**Forced HNSW**	**FP16 (halfvec)**	**Index Scan**	**753 hits**	**0.397 ms**	**0.405 ms**	**9.28% fewer buffer hits**
Stage 4 (10,000c)	**Exact Search**	**FP32 (vector)**	**Seq Scan**	**30,064 hits**	**54.471 ms**	**54.487 ms**	Baseline
Stage 4 (10,000c)	**Exact Search**	**FP16 (halfvec)**	**Seq Scan**	**2,000 hits**	**7.792 ms**	**7.810 ms**	**93.35% fewer buffer hits (15.03x)**
Stage 4 (10,000c)	**Forced HNSW**	**FP32 (vector)**	**Index Scan**	**930 hits**	**0.615 ms**	**0.624 ms**	Baseline
Stage 4 (10,000c)	**Forced HNSW**	**FP16 (halfvec)**	**Index Scan**	**847 hits**	**0.495 ms**	**0.503 ms**	**8.92% fewer buffer hits**

Figure 8: Diagnostic Query 0 (Q0) Shared-Buffer Accesses and Server Execution Time (chart2_q0_diagnostics.csv)

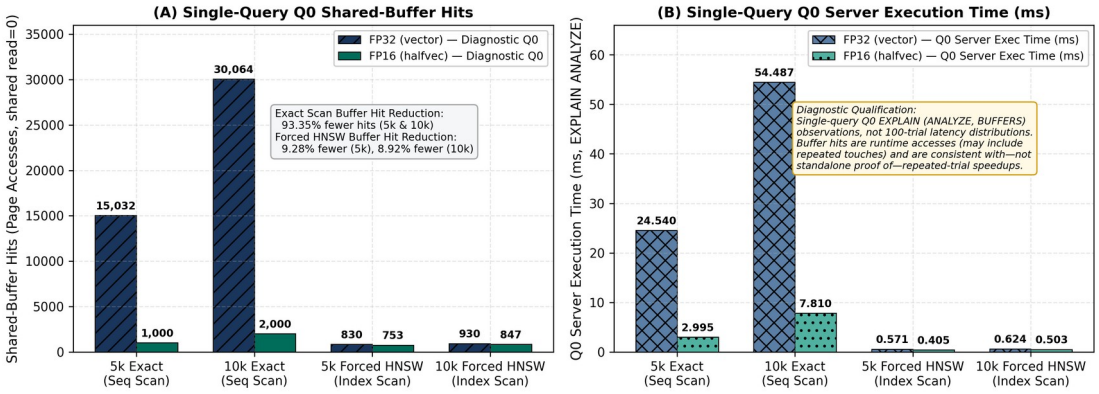


Figure 8: Single-query Query 0 (Q0) diagnostic observations from EXPLAIN (ANALYZE, BUFFERS) (chart2_q0_diagnostics.csv).

Table 7.7 shows how PostgreSQL's query planner (enable_seqscan = on) selected execution paths across representations. Because PostgreSQL's sequential scan cost estimator uses the main table heap's relpages without directly counting out-of-line pg_toast blocks, FP32's forced Seq Scan + Sort cost (202.57 at 5k across 32 heap blocks; 405.12 at 10k across 64 heap blocks) appears smaller to the planner than FP16's forced Seq Scan + Sort cost (1170.57 at 5k across 1,000 inline blocks; 2341.12 at 10k across 2,000 inline blocks). On the dedicated representation tables, **FP32 already selected Index Scan at 1,001 chunks** and continued to use Index Scan at 5,000 and 10,000 chunks. In contrast, the tested **FP16** representation selected Seq Scan at **1,001 chunks** (201 inline heap blocks, cost 235.17) and transitioned to Index Scan using the HNSW index at **5,000 chunks** (Limit cost 529.00 < 1170.57) and **10,000 chunks** (Limit cost 566.31 < 2341.12).

Table 7.7: PostgreSQL query optimizer cost estimates and representation-specific plan selection across scales.

Corpus Scale	Vector Type	Main Heap Blocks	Forced Seq Scan + Sort Cost	HNSW Index Scan + Limit Cost	Planner-Selected Path (enable_seqscan=on)
180 Chunks (Baseline)	FP32 (Monolithic)	< 75 blocks	10.58..10.58 (Seq: 8.25)	365.35..365.79	Seq Scan (10.58 < 365.79)
1,001 Chunks (Stage 2/3)	FP32 (vector)	7 blocks (56.0 KiB)	41.14..41.17 (Seq: 19.51)	Not recorded in Stage 3 JSON	**Index Scan** (already selected at 1k)
1,001 Chunks (Stage 2/3)	FP16 (halfvec)	201 blocks (1.57 MiB)	235.14..235.17 (Seq: 213.51)	Not recorded (> 235.17 est.)	**Seq Scan** (201 inline heap blocks)
5,000 Chunks (Stage 4)	FP32 (vector)	32 blocks (256.0 KiB)	202.55..202.57	35.82..56.17	Index Scan (56.17 < 202.57)
5,000 Chunks (Stage 4)	FP16 (halfvec)	1,000 blocks (7.81 MiB)	1170.55..1170.57 (Seq: 1062.50)	511.82..529.00	**Index Scan (transitioned by 5k: 529.00 < 1170.57)**
10,000 Chunks (Stage 4)	FP32 (vector)	64 blocks (512.0 KiB)	405.10..405.12	68.10..88.43	Index Scan (88.43 < 405.12)
10,000 Chunks (Stage 4)	FP16 (halfvec)	2,000 blocks (15.63 MiB)	2341.10..2341.12 (Seq: 2125.00)	548.68..566.31	**Index Scan (566.31 < 2341.12)**

7.5 RESOURCE COST ANALYSIS, PRACTICAL IMPLICATIONS, AND LIMITATIONS

No financial costing or cloud-pricing evaluation was performed; cost analysis is evaluated strictly via measured system resource consumption (disk storage, buffer page touches, and execution time). At 10,000 chunks, switching from vector(768) to halfvec(768) reduces inclusive relation space by 43.85 MiB (79.20 to 35.35 MiB), HNSW index space by 19.44 MiB (38.89 to 19.45 MiB), and mean exact-scan latency across 100 timed trials by 76.703 ms/query (89.004 to 12.301 ms), while recording 28,064 fewer shared-buffer hits (30,064 vs. 2,000) in the single-query Q0 EXPLAIN (ANALYZE, BUFFERS) diagnostic (reflecting that Q0 run rather than a per-trial guarantee across all 100 trials).

Four methodological validity boundaries and source-validation notes govern these findings:

- **Internal, Construct, and External Validity: Internal validity:** Stage 3 (per-trial transaction/GUC block) and Stage 4 v2 (cur.execute() + cur.fetchall() with GUCs outside the timer; superseding Stage 4 v1's 76.765 ms at 5k and 157.901 ms at 10k) use different timing boundaries; each Stage 4 configuration's 100 trials are repeated runs of the same 20 queries (20 × 5 passes) without confidence intervals or significance tests, and alternating FP32/FP16 order reduces cache-order bias without eliminating all warm-cache or OS effects. **Construct validity:** Recall@K and Agreement@K measure set overlap (not semantic relevance, human qrels, or RAG answer quality), and planner costs are model units rather than elapsed times. **External validity:** results reflect one laptop, one 768-D model (nomic-embed-text), fixed HNSW settings (m = 16, ef_construction = 64, ef_search = 40), and two Stage 4 v2 sizes (5k, 10k) inside 128 MB shared_buffers, so no asymptotic scaling law or exact planner crossover point between 1,001 and 5,000 chunks is claimed (halfvec(1536) = 3,080 B still exceeds ~2 KiB).
- **Index-Build, I/O, and Memory Evidence:** HNSW build times (0.2334 s vs. 0.1606 s at 1k; 1.4615 s vs. 1.2640 s at 5k; 3.7617 s vs. 2.6411 s at 10k) are single-run observations without WAL or peak-RAM profiling. Single-query Q0 plans recorded shared read = 0; shared hit counts page accesses (including repeated touches) and does not prove physical disk I/O.
- **Top-K Latency Parameter & Stage 2 Recall@3 = 0.9833:** Stage 2–4 markdown latency tables omit a K header (not assumed from Review 2's K = 3 or Q0 LIMIT 10 plans), while src/scale_experiment.py (run_scale_benchmark), src/representations.py, and src/hnsw_experiment.py verify top_k = max(top_k_values) = 10 in timed loops, distinct from K in {3, 5, 10} fidelity runs. Stage 2 FP16 exact Recall@3 = 0.9833 equals 59/60 set overlap; the Stage 2 report's phrase 'minor order perturbation' is not established by set intersection, and per-query ID JSONs (benchmark_scale_10k_v2.json) are unbundled.
- **Numeric-Source & Rounding Discrepancies:** (a) pg_column_size(vector(768)) is 3,076 B empirically (49.80% reduction to 1,544 B) vs. 3,080 B by the 4d + 8 formula (49.87%); (b) 1,001-chunk post-index inclusive pg_total_relation_size and unforced Stage 3 planner costs were unrecorded; (c) 1k HNSW build reduction is 31.18% in stage3_hnsw_benchmark.md (31.19% from rounded 0.2334 s / 0.1606 s) and 10k forced-HNSW median difference is 11.86% in chart4_latency.csv vs. 11.84% in Stage 4 prose; and (d) older summaries list PostgreSQL 18.0, pgvector 0.5.0, and work_mem = 64 MB (vs. 18.4, 0.8.4, 4 MB, and maintenance_work_mem = 64 MB in Stage 1–4 reports) and differ on Ollama version (0.4.7 vs. 0.32.13).

8. CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

In this project, we built a reproducible, multi-database benchmarking pipeline in PostgreSQL 18.4 and pgvector 0.8.4 to evaluate how 16-bit half-precision coordinate quantization (halfvec(768)) affects storage layout, HNSW index size, query latency, and top-K retrieval fidelity on MS MARCO passage embeddings. Converting 768-dimensional nomic-embed-text vectors from 32-bit vector(768) (3,076 B measured) to 16-bit halfvec(768) (1,544 B measured) reduced datum size by 49.80% and brought every tuple below PostgreSQL's ~2 KiB TOAST_TUPLE_THRESHOLD. Within our dedicated vector-only representation tables, this eliminated 100% of out-of-line TOAST data payloads (0 bytes, leaving only an empty 8 KiB TOAST index shell), reduced pre-HNSW base relation storage by **60.11%** at 1,001 chunks, reduced inclusive post-index relation storage by **55.35%** at 5,000 chunks and **55.37%** at 10,000 chunks, and reduced standalone HNSW index size by **49.90%–49.98%**.

Across 100 timed trials per configuration under the Stage 4 v2 harness, the inline FP16 representation achieved a **6.22x lower mean latency (7.12x lower median)** in exhaustive exact sequential search at 5,000 chunks and a **7.24x lower mean latency (8.03x lower median)** at 10,000 chunks; the **93.35% reduction** in shared-buffer hits observed in the single-query Q0 diagnostic is consistent with this measured latency improvement, without independently proving per-trial causality. For approximate search (ef_search = 40), forced HNSW latencies were broadly similar between FP32 and FP16 (3.372–3.856 ms FP16 vs. 3.674–4.129 ms FP32 mean), and the PostgreSQL planner automatically transitioned on the FP16 table from Seq Scan at 1,001 chunks (201 heap blocks) to Index Scan at 5,000 and 10,000 chunks (1,000 and 2,000 heap blocks). Finally, FP16 HNSW maintained **0.9600 (96.0%) Recall@10** against uncompressed FP32 exact ground truth at both 5,000 and 10,000 chunks, along with **0.9850** (5k) and **0.9500** (10k) pairwise top-10 candidate agreement with FP32 HNSW.

8.2 FUTURE ENHANCEMENTS

- **1. Scaling to 100k–1M+ Chunks Under Memory Pressure:** Extend the corpus to 100,000–1,000,000+ vectors under constrained shared_buffers and OS cache to measure physical disk reads (shared read > 0) when the FP32 HNSW index exceeds memory before the half-sized FP16 index.
- **2. Repeated Index-Build Trials and WAL Instrumentation:** Run multi-trial CREATE INDEX benchmarks across varying maintenance_work_mem and parallel worker settings while recording Write-Ahead Log (pg_stat_wal) volume and peak memory usage.
- **3. Higher-Dimensional Models and Alternative Quantization Methods:** Evaluate 1,536- and 3,072-dimensional embeddings alongside Scalar Quantization (SQ), Product Quantization (PQ) [7], anisotropic quantization [8], and binary quantization (bit) where halfvec exceeds 2 KiB.
- **4. End-to-End RAG Generation and Relevance Evaluation:** Connect retrieval to official MS MARCO relevance judgments (qrels MRR@10 / nDCG@10) [3] and a local LLM generator (such as the Qwen stage planned in Review 2) to evaluate RAG answer quality [10].

9. REFERENCES

[1] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Chapter 66 — Database Physical Storage, Section 66.2 — TOAST (The Oversized-Attribute Storage Technique). URL: https://www.postgresql.org/docs/18/storage-toast.html

[2] Kane, A., & pgvector Contributors. pgvector: Open-source vector similarity search for Postgres. GitHub Repository. URL: https://github.com/pgvector/pgvector

[3] BeIR / MS MARCO Dataset Repository. Microsoft Machine Reading Comprehension (MS MARCO) passage ranking corpus distributed via Hugging Face Datasets (BeIR/msmarco, configuration 'corpus', split 'corpus'). URL: https://huggingface.co/datasets/BeIR/msmarco

[4] Ollama & Nomic AI. Ollama local model runtime serving nomic-embed-text (768-dimensional dense text embedding model). URL: https://ollama.com/

[5] Pradhan, Y., & Pandey, A. (2026). Compression-Aware Vector Database Indexing for Memory-Efficient and Low-Latency Retrieval-Augmented Generation (RAG): Project Benchmark Codebase and Experimental Archive (BCSE302P — Database Systems Lab, VIT Vellore). URL: https://github.com/yoswinner/CompressionAwareRag.git

[6] Malkov, Y. A., & Yashunin, D. A. (2020). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[7] Jégou, H., Douze, M., & Schmid, C. (2011). "Product Quantization for Nearest Neighbor Search." IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[8] Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., & Kumar, S. (2020). "Accelerating Large-Scale Inference with Anisotropic Vector Quantization." Proceedings of the 37th International Conference on Machine Learning, Proceedings of Machine Learning Research, 119, 3887–3896. https://proceedings.mlr.press/v119/guo20h.html

[9] Aumüller, M., Bernhardsson, E., & Faithfull, A. (2020). "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms." Information Systems, 87, Article 101374. https://doi.org/10.1016/j.is.2019.02.006

[10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems, 33, 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

APPENDIX A — SAMPLE CODE

This appendix provides representative Python and SQL extracts from src/preprocessing.py, src/representations.py, and src/scale_experiment.py that implement deterministic chunking, disaggregated storage auditing, top-K overlap calculation, and the 20-query evaluation benchmark.

Listing A.1: Deterministic Sliding-Window Text Chunking (src/preprocessing.py)

```
def chunk_text(text: str, chunk_size: int = 300, overlap: int = 50) ->
list[str]:
    if not text or not text.strip():
        return []
    text = text.strip()
    if len(text) <= chunk_size:
        return [text]
    chunks, start, step = [], 0, chunk_size - overlap
    while start < len(text):
        chunks.append(text[start : start + chunk_size])
        start += step
    return chunks
```

Listing A.2: Fork-Reconciled Disaggregated Storage Catalog Query (src/scale_experiment.py)

```
SELECT pg_relation_size(c.oid) AS heap_bytes,
       COALESCE(pg_relation_size(c.reltoastrelid), 0) AS toast_table_bytes,
       COALESCE(pg_indexes_size(c.reltoastrelid), 0) AS toast_index_bytes,
       pg_indexes_size(c.oid) AS table_indexes_bytes,
       pg_total_relation_size(c.oid) AS total_inclusive_bytes,
       pg_relation_size(c.oid, 'fsm') + pg_relation_size(c.oid, 'vm') AS
heap_aux_forks_bytes
FROM pg_class c WHERE c.relname = %s;
```

Listing A.3: Top-K Set-Overlap Function & Representative Query Excerpt (src/scale_experiment.py & src/representations.py)

```
def calculate_overlap_recall(ref_ids: list[int], cand_ids: list[int], top_k:
int) -> float:
    if not ref_ids or top_k <= 0:
        return 0.0
    ref_set = set(ref_ids[:top_k])
    cand_set = set(cand_ids[:top_k])
    return len(ref_set.intersection(cand_set)) / float(top_k)

# Representative excerpt from EVALUATION_QUERIES (full 20-query list defined
in src/representations.py):
EVALUATION_QUERIES = [
    'what was the purpose of the manhattan project',                  # Q00
(Diagnostic Q0)
    'how does photosynthesis work in green plants',                    # Q04
    'how do vector databases store and index embeddings',              # Q08
    'difference between supervised and unsupervised machine learning',  # Q09
    # ... [Q01–Q03, Q05–Q07, Q10–Q18 omitted here; see
src/representations.py] ...
    'structure and biological function of human deoxyribonucleic acid dna' #
Q19
]
```

APPENDIX B — TANGIBLE OUTCOME — PUBLICATION/PATENT

A project-related Invention Disclosure Form (IDF) has been submitted through the relevant institutional process. At the time of preparing this report, confirmation and filing details for a formal patent application were not available in the project materials reviewed. Submission of the IDF is acknowledged as an invention-disclosure outcome and should not be interpreted as confirmation that a patent application has been filed or granted.

The project has also produced tangible engineering and research deliverables, including the implemented benchmarking pipeline, automated tests, reproducible dataset caches, experimental reports, numerical result files, and comparative visualizations documenting the evaluation of FP32 and FP16 vector representations:

- **1. Modular Compression-Aware Benchmarking Codebase (src/):** Ten modular Python source files (config.py, database.py, preprocessing.py, embeddings.py, ingestion.py, search.py, benchmark.py, representations.py, hnsw_experiment.py, and scale_experiment.py) implementing deterministic MS MARCO passage streaming, sliding-window chunking, local Ollama nomic-embed-text embedding generation, PostgreSQL vector(768) and halfvec(768) table management, HNSW indexing, and multi-mode latency/plan benchmarking.
- **2. Automated Multi-Database Verification Suite (tests/):** An automated pytest regression suite of **43 passing unit and integration tests** (test_basic.py, test_ingestion_cache.py, test_representations.py, test_hnsw_experiment.py, and test_scale_experiment.py) enforcing fail-closed database isolation and mathematical metric verification.
- **3. Cryptographically Validated Dataset Caches & Manifests (data/raw/):** Local deterministic TSV caches and JSON provenance manifests for the Stage 1–3 1,001-chunk cache (msmarco_1k_chunks.meta.json: target_chunks = 1000, actual_chunks = 1001 across 550 documents) and the Stage 4 canonical streaming cache (msmarco_10k_canonical.meta.json, SHA-256 cdca54aa145a8343916447f9b9fcd8ddcfdb72a3129c5263f5fba3c485434e8b: target_chunks = 10000, actual_chunks = 10001 cached across 5,457 documents before zero-overshoot ingestion and benchmarking of the exact **5,000-chunk** and **10,000-chunk** Stage 4 evaluation corpora).
- **4. Stage-Specific Benchmark Reports & CSV Data Assets (docs/):** Five stage-specific technical benchmark reports (baseline_benchmark.md, stage1_dataset_scaling.md, stage2_fp16_evaluation.md, stage3_hnsw_benchmark.md, and stage4_scale_benchmark.md), cross-stage synthesis documentation (final_results_synthesis.md), and five verified CSV data files (chart1_storage.csv, chart2_q0_diagnostics.csv, chart3_hnsw_recall.csv, chart3_pairwise_agreement.csv, and chart4_latency.csv).